COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010	**SECTION:**	CPE21S2
DATE PERFORMED:	7/31/26	**DATE SUBMITTED:**	8/4/26
NAME:	CRUZ, ELIJAH J.	**INSTRUCTOR:**	SIR QUEJADO

ACTIVITY NO. 4 STACKS

1. PROCEDURE OUTPUT

ILO A & B:

TASKS ILO A

- Test the stack STL operations through the given code below. Provide screenshot per operation, your observations and any other remarks in the output section of your activity template.

MAIN

```
#include <iostream>
#include <stack>

int main() {

   std::stack<int> stack1, stack2;

   //stack 1
   std::cout << "Testing the Stack STL: " <<std::endl;

   //isEmpty
   std::cout << "The Stack is currently empty?: " << stack1.empty() << std::endl;

   //Emplace
   std::cout << "You a newly created stack added: " << stack1.emplace(11) << std::endl;

   //Push
   stack1.push(10);
   std::cout << "The top of the stack is: " << stack1.top() << std::endl;
   stack1.push(9);
   std::cout << "The top of the stack is: " << stack1.top() << std::endl;
   stack1.push(8);
   std::cout << "The top of the stack is: " << stack1.top() << std::endl;
   stack1.push(7);
   std::cout << "The top of the stack is: " << stack1.top() << std::endl;
   stack1.push(6);
   std::cout << "The top of the stack is: " << stack1.top() << std::endl;

   //Pop
   std::cout << "Pop testing: " << std::endl;

   std::cout << "The current size of the stack is: " << stack1.size() << std::endl;

   stack1.pop();
   std::cout << "The top of the stack is: " << stack1.top() << std::endl;


   std::cout << std::endl;

   // Stack 2

   //isEmpty
   std::cout << "The Stack is currently empty?: " << stack2.empty() << std::endl;
```

```
  //Push
   stack2.push(5);
   std::cout << "The top of the stack is: " << stack2.top() << std::endl;
   stack2.push(4);
   std::cout << "The top of the stack is: " << stack2.top() << std::endl;
   stack2.push(3);
   std::cout << "The top of the stack is: " << stack2.top() << std::endl;
   stack2.push(2);
   std::cout << "The top of the stack is: " << stack2.top() << std::endl;
   stack2.push(1);
   std::cout << "The top of the stack is: " << stack2.top() << std::endl;

   //Emplace
   std::cout << "You a newly created stack added: " << stack2.emplace(75) << std::endl;

  //Pop
   std::cout << "The current size of the stack is: " << stack2.size() << std::endl;

   std::cout << "Pop testing: " << std::endl;

   stack2.pop();
   std::cout << "The top of the stack is: " << stack2.top() << std::endl;

   std::cout << "Swap Testing: " << std::endl;
   stack1.swap(stack2);
   stack2.swap(stack1);

  //Display
   std::cout << "Stack 1: ";
       while (!stack1.empty()) {
           std::cout << stack1.top() << " ";
           stack1.pop();
       }

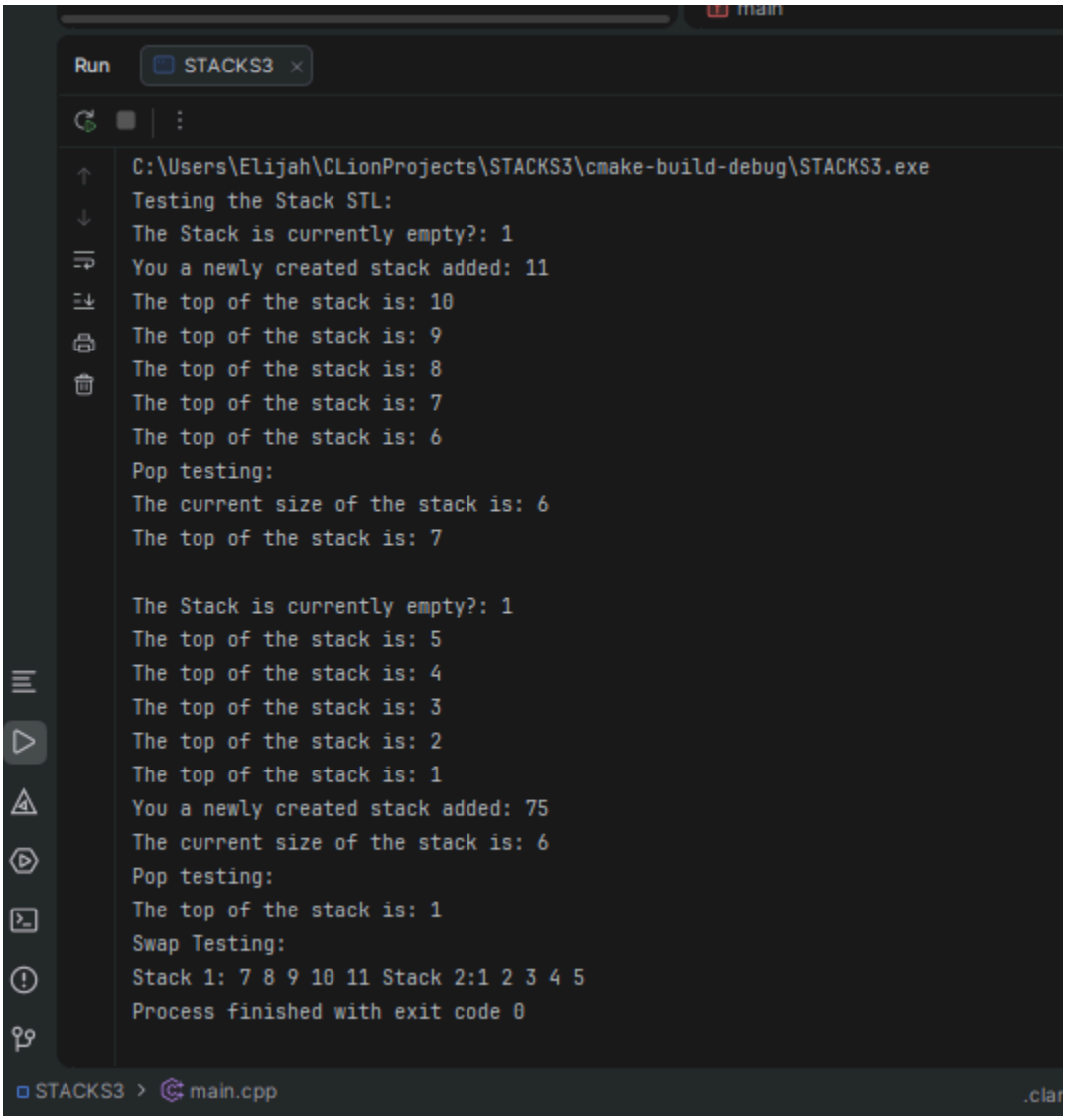

  //Display
   std::cout << "Stack 2:";
       while (!stack2.empty()) {
           std::cout << stack2.top() << " ";
           stack2.pop();
       }



   return 0;

}
```

OUTPUT



TASKS ILO B

- Modify the code given above to include a function that will display all elements in the stack.
- Provide a description of each operation provided.
- Include your output in the output section of your activity template.

MAIN

```
#include <iostream>
#include <stack>
#include "stackList.h"

int main () {

   std::cout << "PUSHING 'C' 'P' 'E' ONTO THE STACK" << std::endl;
```

```
    //push
    push('C');
    push('P');
    push('E');

    //display the current top
    Top<char>(); //top should be 'E'
    displayALL<char>();
    std::cout<<"\nPOPPING ALL THE ELEMENTS"<<std::endl;

    while (!isEmpty<char>()) {
        std::cout<<"POPPED: "<<pop<char>() <<std::endl;
    }

    std::cout<<"\nTESTING THE UNDERFLOW"<<std::endl;
    pop<char>();

    return 0;
}
```

HEADER FILE

```
#ifndef STACK_LIST_H
#define STACK_LIST_H

#include <iostream>

//class node that contains data and the next
template <typename T>
class Node {
public:
    T data; //VALUE THAT WE NEED TO STORE
    Node<T>* next; // POINTER TO THE NEXT NODE
};

//global pointer for the top of the stacl
template <typename T>
Node<T> *head = nullptr;

//operations for the stack
// push
template <typename T>
void push(T newData) {
    //create a new node and assign the newData
    Node<T>* newNode = new Node<T>;
    newNode->data = newData;
    newNode->next = head<T>;
    head<T> = newNode;
}
//pop
template <typename T>
T pop() {
    if (head<T> == nullptr) {
        std::cout<<"Stack Underflow"<<std::endl;
        return T();
    }
    //save the current top node
    Node<T>* temp = head<T>;
    //store its data
    T tempVal = temp->data;

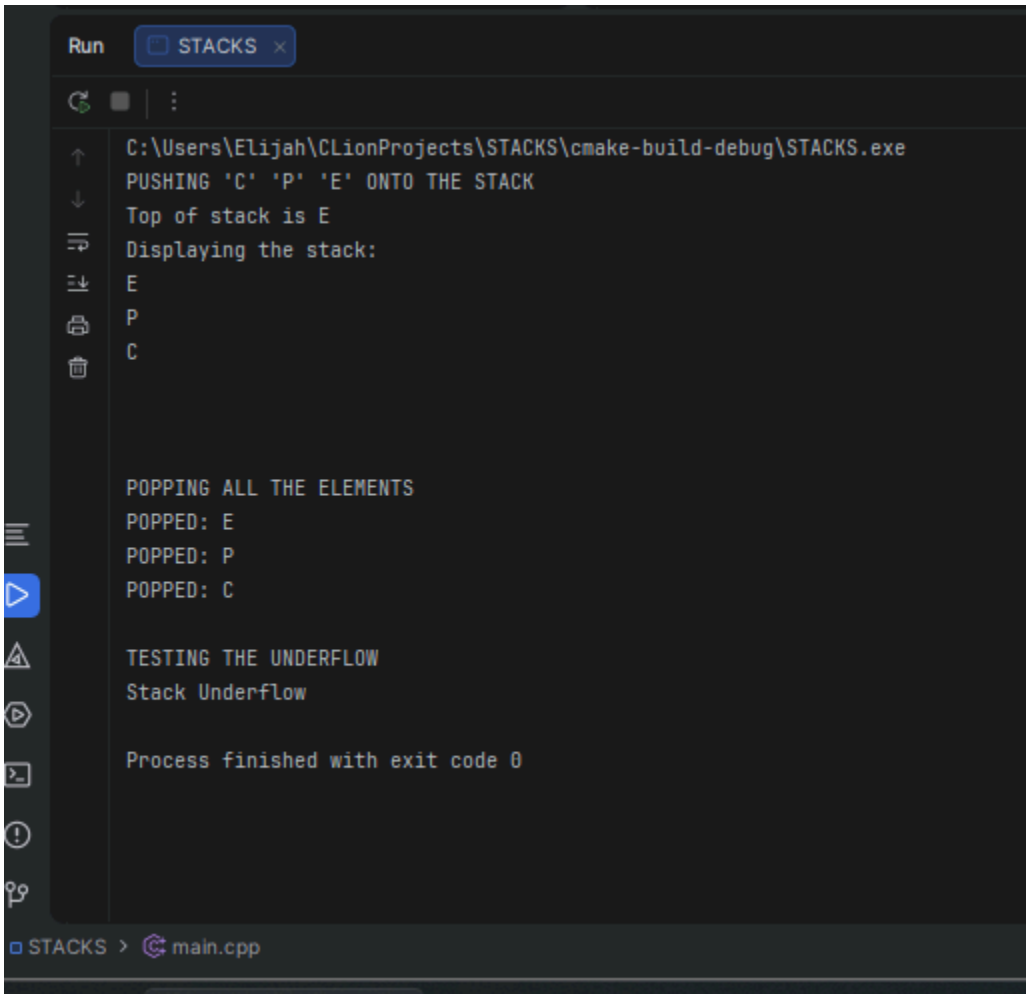
    //move the head to the next node
    head<T> = temp->next;
    //free the removed node
    delete temp;
    //return the removed value
    return tempVal;
}
// isEmpty
```

```
template <typename T>
bool isEmpty() {
   return head<T> == nullptr;
}

// top
template <typename T>
void Top() {
   //error catching:
   if (isEmpty<T>()) {
       std::cout<<"Stack Underflow"<<std::endl;
       return;
   }
   //display the Top node
   std::cout<<"Top of stack is "<<head<T>->data<<std::endl;
}

//display All
template <typename T>
void displayALL() {
   if (head<T> == nullptr) {
       std::cout<<"Stack Underflow\n"<<std::endl;
       return;
   }
   std::cout<<"Displaying the stack: \n";
   Node<T>* temp = head<T>;
   while (temp != nullptr) {
       std::cout<<temp->data<<" \n";
       temp = temp->next;
   }
   std::cout<< "\n"<<std::endl;
}
#endif
```

OUTPUT



2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Problem Definition:
Stacks can be used to check whether the given expression has balanced symbols. This algorithm is very useful in compilers. Each time the parser reads one character at a time. If the character is an opening delimiter such as (, {, or [- then it is written to the stack. When a closing delimiter is encountered like), }, or]-the stack is popped. The opening and closing delimiters are then compared. If they match, the parsing of the string continues. If they do not match, the parser indicates that there is an error on the line.

Steps:

1. Create a Stack.
2. While(end of input is not reached) {
   If the character read is not a symbol to be balanced, ignore it.
   If the character is an opening symbol, push it onto the stack.
   If it is a closing symbol:
   I. Report an error if the stack is empty.
   II. Otherwise, pop the stack. d) If the symbol popped is not the corresponding opening symbol,

report an error.

3. At the end of input, if the stack is not empty: report an error.

CODE:

```cpp
//main
#include <iostream>
#include <string>
#include "stackList.h"

bool isBalanced(std::string input) {

   while (!isEmpty<char>()) {
       pop<char>();
   }
   for (int i = 0; i < input.length(); i++) {
       char c = input[i];

       //opening symbols
       if (c == '(' || c == '[' || c == '{') {
           push<char>(c);
       }

       //closing symbols
       else if (c == ')' || c == ']' || c == '}') {

           if (isEmpty<char>()) {
               std::cout << "ERROR: '" << c << "' has nothing to match" << std::endl;
               return false;
           }

           char popped = pop<char>();

           if ((c == ')' && popped != '(') ||
               (c == ']' && popped != '[') ||
               (c == '}' && popped != '{')) {
           std::cout << "ERROR: '" << popped << "' does not match '" << c << "'" << std::endl;
           return false;
               }
       }

   }

   if (!isEmpty<char>()) {
       std::cout << "ERROR: symbol left open at the end" << std::endl;
       return false;
   }

   return true;
}

int main() {

   std::string test1 = "(A+B)+(C-D)";
   std::string test2 = "((A+B)+(C-D)";
   std::string test3 = "((A+B)+[C-D])";
   std::string test4 = "((A+B]+[C-D]}";

   std::cout << "TESTING: " << test1 << std::endl;
   std::cout << (isBalanced(test1) ? "VALID" : "INVALID") << std::endl;

   std::cout << "\nTESTING: " << test2 << std::endl;
   std::cout << (isBalanced(test2) ? "VALID" : "INVALID") << std::endl;

   std::cout << "\nTESTING: " << test3 << std::endl;
   std::cout << (isBalanced(test3) ? "VALID" : "INVALID") << std::endl;

   std::cout << "\nTESTING: " << test4 << std::endl;
```

```
    std::cout << (isBalanced(test4) ? "VALID" : "INVALID") << std::endl;

    return 0;
}
```

HEADER FILE

```
#ifndef STACK_LIST_H
#define STACK_LIST_H
#include <iostream>

// Class node that contains data and the next pointer
template <typename T>
class Node {
public:
    T data;
    Node<T>* next;
};

// Pointer for the top of the stack
template <typename T>
Node<T>* head = nullptr;

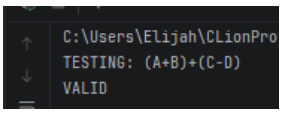
// Push
template <typename T>
void push(T newData) {
    Node<T>* newNode = new Node<T>;
    newNode->data = newData;
    newNode->next = head<T>;
    head<T> = newNode;
}

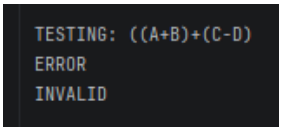
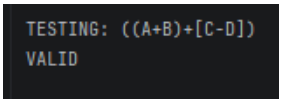
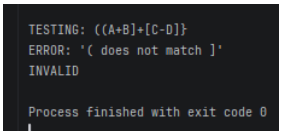
// Pop
template <typename T>
T pop() {
    if (head<T> == nullptr) {
        std::cout << "Stack Underflow" << std::endl;
        return T();
    }
    Node<T>* temp = head<T>;
    T tempVal = temp->data;
    head<T> = temp->next;
    delete temp;
    return tempVal;
}

// isEmpty operation
template <typename T>
bool isEmpty() {
    return head<T> == nullptr;
}

#endif
```

SELF CHECKING (OUTPUT)

Expression	Valid? (Y/N)	Output (Console Screenshot)	Analysis
(A+B)+(C-D)	YES		As the program reads every string in the expression there is a

			match, thus ending the program with a cleared out stack
((A+B)+(C-D)	NO		The program reads the first character '(' and finds a pair for the following characters. However, since the expression is lacking a ')' it returns it as invalid.
((A+B)+[C-D])	YES		As said, the program reads again every string in the expression and finds a match, thus ending the program with the stack cleared
((A+B]+[C-D])	NO		Contains a mismatched character upon reading the entire expression. (is not the same character as].

CONCLUSIONS/LEARNINGS

For this project, we used a stack to check whether the symbols in an expression, like parentheses, brackets, and braces, are properly balanced. The program reads the expression one character at a time, pushing every opening symbol onto the stack and popping it off when a matching closing symbol appears. We tested this on several expressions, including a fully valid one at the given procedures and supplementary activity, one with an opening symbol that was never closed, one with mixed but properly nested brackets, and one where the bracket types didn't match. In every case, the program correctly identified whether the expression was balanced, showing us that a stack is a strong fit for this kind of matching problem.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **Gemini**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Grammar correction, code assistance**

COMPUTER ENGINEERING

QUEZON CITY

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **Explanation of stacks and list and how to implement them.**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.